OpenHarmony-Audit-Agent 技术方案与创新性报告

本报告描述统一的漏洞挖掘方法与工程架构。报告中的 OpenHarmony
场景为抽象化示例，不包含未披露的真实漏洞、真实 PoC 或虚构实验结果。

摘要

OpenHarmony 源码漏洞挖掘同时面临规模大、跨模块调用链复杂、静态分析保守近似和漏洞证据难以复现等问题。本项目将一个可配置的多阶段审计工作流与 CodeQL-AI 的静态告警后处理能力统一为证据驱动的分层分析系统。静态分析负责高召回候选生成，角色化审计负责入口、数据流、生命周期和反证分析，CodeQL-AI
负责误报经验匹配、清洗函数识别、QL 优化和路径可达性复核，最终通过统一证据模型输出结构化漏洞报告。

1. 问题定义与比赛目标

比赛要求工具能够分析 OpenHarmony 开源代码，输出漏洞描述、CWE/CVE、可执行
PoC、影响范围、修复建议和完整报告，同时关注准确性、完整性、PoC 质量、版本影响、性能和创新性。本项目把这些要求转化为可追溯的证据对象，避免单一模型结论直接成为漏洞结论。

2. 统一理论模型

静态分析通常采用保守近似，适合提高召回率，却会把未建模的 sanitizer、分支约束和跨函数语义折叠成告警。审计工作流把一个告警拆成入口、信任边界、逐跳调用链、危险操作和缺失不变量；CodeQL-AI
再对路径中的清洗逻辑和分支可达性进行针对性复核。

对候选 c，系统维护证据集合 $E(c)$。最终决策由证据门禁产生：

STATIC_ONLY = static_path ∧ invariant ∧ counter_evidence

CONFIRMED = STATIC_ONLY ∧ independent_verification

REJECTED = STATIC_PATH ∧ decisive_counter_evidence

NEEDS_EVIDENCE = otherwise

模型 confidence
只描述推理质量，不替代证据门禁。误报经验默认以低置信度保存，经回归验证后才允许提升状态。

3. 系统架构

系统包含五个层次：目标与任务层、静态发现层、语义审计层、证据复核层和报告层。工作流控制器按 analysis unit 分片，冻结任务包和 checkpoint；CodeQL-AI provider 通过稳定接口提供 SARIF 解析、FP experience、QL 优化、sanitizer 分析和符号验证。

架构图见 docs/architecture.mmd。

4. 统一数据模型

对象 · 作用 · 关键字段 ·

`AuditCampaign` · 冻结目标和威胁模型 · snapshot、revision、units、interfaces ·

`Finding` · 归一化静态告警 · rule、CWE、source、sink、codeFlow ·

`CandidatePacket` · 冻结漏洞候选 · invariant、callChain、impact、counterEvidence ·

`Evidence` · 描述可复核事实 · kind、producer、inputHash、provenance ·

`Decision` · 输出门禁结果 · verdict、rationale、evidenceRefs ·

`ExperienceRecord` · 跨任务复用知识 · fpType、rootCause、confidence、validation ·

5. 统一工作流

5.1 Inventory

读取抽象化目标 profile，固定代码快照、分析单元、关注面、攻击者和信任边界。

5.2 Discovery

运行 CodeQL 或导入已有 SARIF，将告警转换为 Finding，保留原始 ruleId、CWE、源点、汇点和 code flow。

5.3 Semantic Audit

entrypoint-boundary 识别入口与攻击者控制；dataflow-lifetime
分析数据传播、容量、对象生命周期和危险操作；counterevidence-history 主动寻找反例；chain-composer 组装漏洞不变量。候选在此阶段冻结，后续模块只能追加证据。

5.4 Evidence Enrichment

FP Experience 负责历史误报匹配，Function-Level Sanitizer 负责发现未建模的清洗函数，QL Optimizer 负责提出查询修补，Symbolic Sanitizer 负责在明确攻击约束下验证路径是否可达。每个模块只产生带 provenance 的 Evidence，不直接改写候选结论。

5.5 Decision 与 Feedback

Blind Verifier 独立重建候选，防止发现阶段的期望结论污染验证。最终报告区分 CONFIRMED、STATIC_ONLY、REJECTED 和 NEEDS_EVIDENCE。确认的误报模式写入经验库，供下一轮 QL 优化使用。

6. 关键技术

6.1 SARIF 到证据对象

归一化器读取 SARIF 2.1.0 的 runs、rules、results、locations 和 codeFlows，并计算原始文件 SHA-256。这样同一个告警可以在不同工作流阶段通过 finding_id 和 input_hash 追踪。

6.2 误报经验闭环

经验包含模式摘要、根因、CWE、查询 ID、状态和验证计数。低置信度经验只提供修补建议，不会直接 suppress 新告警；当回归结果支持经验时，再提升为 active 状态。

6.3 聚焦式符号验证

Symbolic Sanitizer 只对 SARIF 路径上的关键分支和攻击输入约束执行局部符号分析，避免全程序状态爆炸。LLM 负责选择分支和构造约束，求解器负责给出可达性结论，从而把语义判断与确定性复核分离。

6.4 双模型运行时

统一 AgentRuntime.run(prompt, workdir, metadata) 接口，分别适配 Claude Code 的 stream-json 和 OpenAI-compatible 的流式 Chat Completions。运行配置只保存模型名、端点和环境变量名，不保存密钥。

7. OpenHarmony 适配方式

首版使用抽象 target profile，不绑定具体版本、设备或真实漏洞。profile 可以描述 IPC、Parcel、NAPI、权限、缓冲区、整数和生命周期等关注面，并为后续代码快照、CodeQL 数据库、构建环境和设备验证后端预留接口。

8. PoC、影响与修复报告

PoC 阶段只生成复现计划、输入约束、测试桩需求和安全边界，不自动执行或提交。报告结构固定包含源码位置、漏洞描述、CWE/CVE、触发条件、调用链、影响模块/版本、PoC 计划、修复建议和证据引用。

9. 与比赛评价维度的对应关系

评价维度 · 对应机制 ·

准确性 · 反证分析、FP experience、sanitizer 识别、blind verification ·

完整性 · CodeQL 高召回扫描、分片 worker、跨组件 unit profile ·

CWE/CVE · 规则映射、结构化 Finding、报告字段约束 ·

PoC 质量 · PoC planner、攻击约束和复现条件模板 ·

影响分析 · snapshot、unit、调用链和 threat model ·

修复建议 · QL patch、sanitizer 逻辑和修复计划证据 ·

性能 · 分片调度、局部符号执行、运行时事件和 checkpoint ·

创新性 · 统一证据模型、模型协作与求解器复核的分层闭环 ·

10. 局限性与披露边界

没有真实构建、设备或动态 PoC 结果时，候选只能标记为 STATIC_ONLY。本项目不自动向 OpenHarmony 社区、CVE 平台或其他第三方提交漏洞。任何真实漏洞、源码片段、设备信息和 API 密钥都应在人工复核和披露流程完成后再进入正式材料。

11. 当前工程状态

代码：统一契约、SARIF 归一化、证据门禁、经验库桥接、CodeQL 适配器、CodeQL-AI provider 和双 runtime 已实现。

报告：本文件是可持续编辑的中文 Markdown 主稿，PDF 由同一内容生成。

数据：当前不包含真实 OpenHarmony 漏洞结论和虚构性能数据。

后续：接入实际代码快照、CodeQL 数据库、设备/仿真验证后端，并填充真实评测结果。

附录：当前运行记录

{

"findingCount": 0,

"decisionCount": 0,

"decisions": []

}